PepPro Group: paid-click attribution is being lost at the research-use gate

Prepared 6 September 2026 for the PepPro development team. Everything below was measured against the live site and the live Whop event stream. Nothing is inferred.

1. What is wrong

Whop / Meta reports **257 paid clicks** into `www.pepprogroup.com`. In PepPro's own Whop event stream over the last 10 days, only **7 of 343 people** carry an ad click identifier at all. So roughly **250 of 257 paid visitors arrive completely unattributable**, and the ads dashboard consequently reports **0 purchases**.

> This is a **measurement** defect, not a sales defect. The dashboard is not failing to record purchases that happened. The site cannot tell paid traffic apart from anyone else in the first place, so no purchase can ever be credited to an ad.

2. The mechanism, measured

An ad points at, for example, `/peptides?fbclid=ABC123`. The site immediately redirects to the research-use gate and folds the **entire** original address, query string included, into a single URL-encoded `next` parameter:

```
/peptides?fbclid=ABC123
        ↓
/login?next=%2Fpeptides%3Ffbclid%3DABC123
```

The click id is not deleted. It is nested. And that is the whole problem, because every consumer that reads tracking parameters reads `location.search`, where the only top-level key is now `next`. Measured in-page on the live site:

```
[...new URLSearchParams(location.search).keys()]   // -> ["next"]
new URLSearchParams(location.search).get('fbclid') // -> null
typeof window.whop                                 // -> "object"   (pixel IS loaded and
firing)
```

The Whop pixel loads and fires normally on that page. It simply has nothing to read. `/login` accounts for **728 of all recorded page hits** in the window, so this is the path essentially all paid traffic takes.

Passing the gate does not repair it. We ticked the confirmation box and clicked Enter on the live site. The gate dismissed and content rendered, but the address stayed on `/login?next=...` and the buried parameters were **still not top-level**. So the click id is unreadable before the gate and unreadable after it. It never becomes visible at any point in the visit.

3. The measured funnel (10 days, 1,182 events, complete paged read)

Segment	People	Past the gate	Product page	Cart	Checkout	Bought
Ad attributed	**7**	3	1	0	0	**0**
Everyone else	336	36	10	0	2	**1**

Whole-site commerce activity in those 10 days: **18 product views, 6 add-to-carts, 1 purchase**. The single order came from non-paid traffic. Campaign spend over the same period was **$432.76** at a 1.79% click-through rate, so the ads are buying clicks; the clicks are simply arriving anonymous.

4. Fix A: minimal, leaves the gate exactly as it is

Roughly ten lines. It does not change routing, does not change what any visitor sees, and does not alter the gate's behaviour. It unwraps `next` and lifts the tracking parameters back to the top level *before* the pixel reads them.

Placement is the whole trick: this must execute **before** the existing `<script id="whop-pixel">` block, because that block calls `whop.track("page")` immediately. In Next.js, render it directly above the pixel, with the same or an earlier loading strategy (`beforeInteractive` is safest).

```
<script id="whop-param-restore">
(function () {
  try {
    var cur  = new URLSearchParams(window.location.search);
    var next = cur.get('next');
    if (!next) return;

    var q = next.indexOf('?');
    if (q === -1) return;

    var inner   = new URLSearchParams(next.slice(q + 1));
    var changed = false;

    inner.forEach(function (value, key) {
      if (!cur.has(key)) { cur.set(key, value); changed = true; }
    });
    if (!changed) return;

    var qs = cur.toString();
    window.history.replaceState(
      null, '',
      window.location.pathname + (qs ? '?' + qs : '') + window.location.hash
    );
  } catch (e) { /* never block the page */ }
})();
</script>
```

It sits immediately above your existing block, which stays untouched:

```
<script id="whop-pixel">
!function(w,d,s,u,n,a,b){ ... }(window,document,"script","https://t.whop.tw","whop");
whop.setScope("biz_oYbMas6NGuxzJp");
whop.track("page");
</script>
```

> **Why this is safe.** It only *adds* keys that are not already present, and it never removes or rewrites `next`, so the gate's own post-entry redirect keeps working unchanged. It is wrapped in try/catch so it can never block page render. If there is no `next` parameter, or `next` carries no query string, it does nothing at all.

How to verify Fix A

```
// Paste in the browser console on an ad-style landing URL, BEFORE and AFTER the fix.
// Load:  https://www.pepprogroup.com/peptides?fbclid=VERIFY123
[...new URLSearchParams(location.search).keys()]
// BEFORE the fix -> ["next"]                  (fbclid invisible, attribution dead)
// AFTER  the fix -> ["next","fbclid"]         (fbclid readable, attribution works)

new URLSearchParams(location.search).get('fbclid')
// BEFORE -> null
// AFTER  -> "VERIFY123"
```

Then confirm end to end: load an ad-style URL, pass the gate, place a test order, and check that the order appears in the Whop dashboard **with a paid source attached** rather than as organic.

5. Fix B: the proper fix, and the one we recommend

Fix A restores measurement. **Fix B restores measurement and fixes the customer experience at the same time**, and it makes Fix A unnecessary.

Stop redirecting to `/login`. Serve the route the visitor actually asked for, and render the research-use confirmation as an **overlay on that page**, gated on a persisted flag.

1. Serve `/peptides?fbclid=ABC123` directly. **The URL never changes**, so the query string stays intact and every analytics consumer reads it normally, with no unwrapping needed anywhere.
2. On first visit, render the same confirmation as a fixed, full-viewport overlay above the page content. Block interaction with the page beneath it until the box is ticked and Enter is pressed.
3. Persist the confirmation (cookie or `localStorage`) exactly as today, so returning visitors are not asked twice.
4. Keep the `/login` route working for anyone who navigates to it directly, and for the email-verification path.

> **The second benefit is commercial, not technical.** Today a visitor clicks an ad for a specific product and is bounced to a URL that is not the product. Ad-to-page mismatch depresses conversion independently of tracking, and platform quality scoring penalises it. Fix B removes that too.

How to verify Fix B

```
// 1. Load an ad-style URL in a fresh private window:
//    https://www.pepprogroup.com/peptides?fbclid=VERIFY123
// 2. The address bar must STILL read /peptides?fbclid=VERIFY123   (it must not become
/login)
// 3. The verification overlay must be visible and must block the page beneath it.
// 4. In the console, before dismissing the overlay:
new URLSearchParams(location.search).get('fbclid')   // -> "VERIFY123"
// 5. Tick, Enter, confirm the overlay is gone and the URL is unchanged.
// 6. Reload. The overlay must not reappear.
```

6. Appendix: a separate, smaller defect in the same codebase

Not part of the two fixes above, but worth handling while someone is in this code. **Page views fire twice on client-side navigation.** This was first reported on 1 September 2026 and is still live: **90 of 716 page events (13%)** in the last 10 days are duplicates with the same person, the same path and the same second, but different event ids.

The effect is that PepPro's traffic figures read roughly 13% high. It does not affect the attribution problem above, and it does not affect person-level counts, which is why the funnel table in section 3 is built on people rather than page views. The usual cause is a page-view call bound to a router event that fires on both the route change and the subsequent render; it should fire once per committed navigation.

7. Summary

	Fix A	**Fix B**
Size	~10 lines, one file	Routing / gate rendering change
Restores attribution	Yes	Yes
Fixes ad-to-page mismatch	No	Yes
Changes what visitors see	No	Yes, the gate becomes an overlay
Recommended	As an immediate stopgap	**Yes, as the real fix**

If Fix B is scheduled soon, apply Fix A now anyway. Every day it is not applied is another day of paid traffic that can never be attributed, and that history cannot be reconstructed later.